



المعهد الوطني للبريد والهواصالات
ᄒᄒᄒᄒᄒᄒ ᄒᄒᄒᄒᄒᄒ ᄒᄒᄒᄒᄒᄒ ᄒᄒᄒᄒᄒᄒ
Institut National des Postes et Télécommunications

INSITUT NATIONAL DES POSTES ET TELECOMMUNICATIONS

APPROCHES ET TECHNOLOGIES DEVOPS RAPPORT

TP2 : Atelier Git/GitHuB

Realisé par :

Tahiri Alaoui NADA
Douae Ghayati

Encadrant :

Pr Asmae RETBI

14 avril 2026

0 Table des matières

1 Introduction	2
2 Rôle SCRUM Master — Mise en place du projet	2
2.1 Création du projet et dépôt local	2
2.2 Premier commit et consultation du journal	2
2.3 Création du dépôt distant sur GitHub et push	3
2.4 Vérification sur GitHub	3
2.5 Ajout de collaborateurs	4
3 Rôle Développeur / Testeur	4
3.1 Clone du dépôt et création d'une branche	4
3.2 Rebase de la branche sur master	4
3.3 Pull Request et revue de code	4
4 Rôle SCRUM Master — Merge et clôture	5
4.1 Merge de la branche feature	5
4.2 Merge local avec visualisation du graphe	5
5 Opérations avancées : Fork et Rebase	5
5.1 Fork	5
5.2 Rebase avancé	6
6 Différences conceptuelles	6
6.1 Fork vs Clone	6
6.2 Merge vs Rebase — Analyse à partir des captures	6
7 Conclusion	8

1 Introduction

Ce rapport présente les travaux pratiques réalisés dans le cadre du devoir n°2 portant sur la gestion de versions avec **Git** et les plateformes **GitHub** et **GitLab**. L'objectif est de mettre en pratique les concepts fondamentaux du workflow collaboratif : initialisation d'un dépôt local, commit, push vers un dépôt distant, gestion des branches, pull requests, ainsi que les opérations avancées de *Fork* et *Rebase*.

2 Rôle SCRUM Master — Mise en place du projet

2.1 Création du projet et dépôt local

La première étape consiste à initialiser un dépôt Git local dans le répertoire du projet, puis à y ajouter les fichiers sources.

Listing 1 – Initialisation du dépôt Git local

```
git init
git add .
git status
git commit -m "Initial commit"
```

La figure suivante illustre ces commandes exécutées dans le terminal VS Code. On observe que le fichier `main.java` est correctement détecté et ajouté à la zone de staging (*index*).

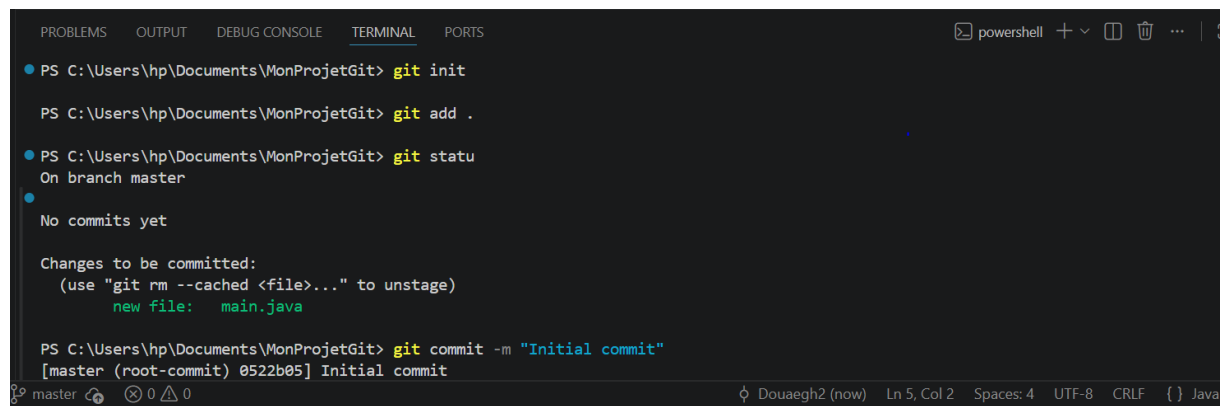
The image is a screenshot of the Visual Studio Code (VS Code) terminal window. The terminal is running a PowerShell session. The user has executed the following commands: 'git init', 'git add .', 'git status', and 'git commit -m "Initial commit"'. The output of 'git status' shows 'On branch master' and 'No commits yet'. It also lists 'Changes to be committed:' with '(use "git rm --cached <file>..." to unstage)' and 'new file: main.java'. The final output shows the commit hash '[master (root-commit) 0522b05] Initial commit'. The terminal interface includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The status bar at the bottom shows 'master', '0' errors, '0' warnings, and the file encoding 'UTF-8'.

FIGURE 1 – Initialisation du dépôt Git et premier commit dans VS Code

2.2 Premier commit et consultation du journal

Après le commit initial, la commande `git log` permet de vérifier l'historique.

```

No commits yet
PS C:\Users\hp\Documents\MonProjetGit> git commit -m "Initial commit"
[master (root-commit) 0522b05] Initial commit
1 file changed, 5 insertions(+)
 create mode 100644 main.java
PS C:\Users\hp\Documents\MonProjetGit> git log
commit 0522b055fda0281877106370f5eaae8fa5c6da3b (HEAD -> master)
Author: Douaegh2 <ghayatidouae17@gmail.com>
Date: Tue Apr 14 10:10:49 2026 +0100

    Initial commit
PS C:\Users\hp\Documents\MonProjetGit>

```

FIGURE 2 – Résultat de `git log` après le premier commit

On voit que le commit 0522b05 (HEAD → master) a bien été enregistré avec le message “*Initial commit*”, l’auteur et la date.

2.3 Création du dépôt distant sur GitHub et push

Un nouveau dépôt est créé sur GitHub. Le dépôt local est ensuite lié au dépôt distant via `git remote add origin`, puis les modifications sont poussées.

Listing 2 – Lien et push vers GitHub

```

git remote add origin https://github.com/Douaegh2/MonProjetGit.git
git branch -M main
git push -u origin main

```

```

PS C:\Users\hp\Documents\MonProjetGit> git remote
e add origin https://github.com/Douaegh2/MonProj(HEAD -> master)
etGit.git
PS C:\Users\hp\Documents\MonProjetGit> git branc
h -M main
PS C:\Users\hp\Documents\MonProjetGit> git push
-u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 312 bytes | 156.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Douaegh2/MonProjetGit.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.

```

FIGURE 3 – Push du dépôt local vers GitHub

Le push s’est effectué avec succès : 3/3 objets transférés, la branche `main` est désormais suivie par `origin/main`.

2.4 Vérification sur GitHub

Le dépôt est visible sur GitHub avec le fichier `main.java` et le commit initial.

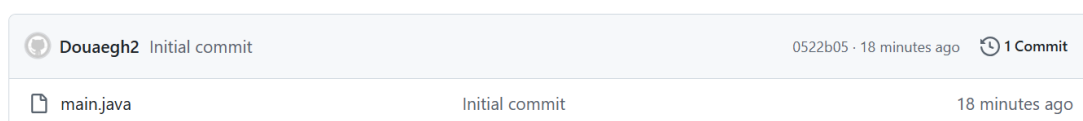


FIGURE 4 – Dépôt GitHub après le push initial

2.5 Ajout de collaborateurs

Via *Settings* → *Collaborators*, une invitation est envoyée au collaborateur (Développeur/Testeur).

```
PS C:\Users\hp\Documents\Atelier-git> git branch
* feature-ajout
  master
PS C:\Users\hp\Documents\Atelier-git> git rebase master
Current branch feature-ajout is up to date.
PS C:\Users\hp\Documents\Atelier-git> git log --graph --oneline
* 6138c52 (HEAD -> feature-ajout, origin/feature-ajout) Ajout feature
* 5840b2a (origin/master, origin/HEAD, master) First change
* 7444d68 Initial commit
PS C:\Users\hp\Documents\Atelier-git> █
```

FIGURE 5 – Gestion des accès — invitation du collaborateur (Pending Invite)

3 Rôle Développeur / Testeur

3.1 Clone du dépôt et création d'une branche

Le développeur accepte l'invitation, clone le dépôt distant dans son espace de travail local, puis crée une nouvelle branche pour isoler ses modifications.

Listing 3 – Clone et création de la branche feature

```
git clone https://github.com/Douaegh2/MonProjetGit.git
git checkout -b feature-ajout
# ... modifications du code Java ...
git add .
git commit -m "Ajout feature"
git push origin feature-ajout
```

3.2 Rebase de la branche sur master

Avant d'ouvrir une pull request, le développeur effectue un rebase de sa branche sur **master** afin d'intégrer les dernières modifications du tronc commun.

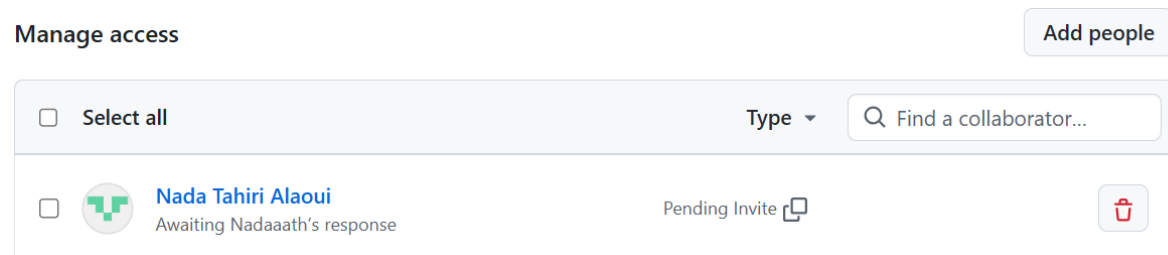


FIGURE 6 – Rebase de feature-ajout sur master et log linéaire

Le log montre un historique **linéaire** : feature-ajout est bien à jour par rapport à origin/master.

3.3 Pull Request et revue de code

Le développeur ouvre une *Pull Request* sur GitHub depuis la branche **feature-ajout** vers **master**, permettant au SCRUM Master de relire et valider le code.

4 Rôle SCRUM Master — Merge et clôture

4.1 Merge de la branche feature

Le SCRUM Master effectue les changements nécessaires puis fusionne la branche `feature-ajout` dans `master` via GitHub.

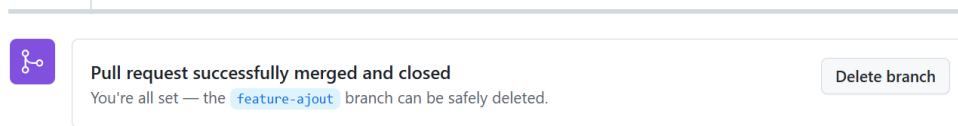


FIGURE 7 – Pull Request fusionnée avec succès sur GitHub

La Pull Request est marquée “*merged and closed*”. La branche `feature-ajout` peut être supprimée sans risque.

4.2 Merge local avec visualisation du graphe

En local, la fusion est réalisée avec `git merge` et le résultat est visualisé avec `git log -graph`.

```
PS C:\Users\hp\Documents\Atelier-git> git log --graph --oneline --decorate --all
* 2fcf4c4 (HEAD -> master) Merge branch 'test-branch-2'
|
| * 4697707 (test-branch-2) Test branch 2
|/
* 4ba7de3 (test-branch) Test branch commit
| * 6138c52 (origin/feature-ajout, feature-ajout) Ajout feature
|/
* 5840b2a (origin/master, origin/HEAD) First change
* 7444d68 Initial commit
PS C:\Users\hp\Documents\Atelier-git> █
```

FIGURE 8 – Graphe de l’historique après le merge de `test-branch-2` dans `master`

On distingue clairement la **structure en diamant** (*merge commit*) caractéristique d’un merge classique : deux branches convergent vers un commit de fusion `2fcf4c4`.

5 Opérations avancées : Fork et Rebase

5.1 Fork

Un *fork* a été réalisé à partir d’un dépôt public afin d’en obtenir une copie indépendante sous notre compte GitHub. Cette opération est effectuée directement depuis l’interface GitHub (bouton *Fork*).

5.2 Rebase avancé

La branche `rebase-branch` a été créée, modifiée, puis rebasée sur `master`.

```
>> git commit -m "Rebase commit"
[rebase-branch 41436c3] Rebase commit
1 file changed, 1 insertion(+), 3 deletions(-)
PS C:\Users\hp\Documents\Atelier-git> git rebase master
Current branch rebase-branch is up to date.
PS C:\Users\hp\Documents\Atelier-git> git log --graph --oneline --decorate --all
* 41436c3 (HEAD -> rebase-branch) Rebase commit
* 2fcf4c4 (master) Merge branch 'test-branch-2'
|\
| * 4697707 (test-branch-2) Test branch 2
|/
* 4ba7de3 (test-branch) Test branch commit
| * 6138c52 (origin/feature-ajout, feature-ajout) Ajout feature
|/
* 5840b2a (origin/master, origin/HEAD) First change
* 7444d68 Initial commit
```

FIGURE 9 – Rebase de `rebase-branch` sur `master` et visualisation avec `git log --graph`

6 Différences conceptuelles

6.1 Fork vs Clone

Fork vs Clone

	Fork	Clone
Définition	Copie côté serveur d'un dépôt distant vers votre propre compte GitHub	Copie locale d'un dépôt (distant ou forké) sur votre machine
Lieu	GitHub / GitLab (cloud)	Machine locale
Lien avec l'original	Connexion GitHub maintenue (possibilité de PR vers l'original)	Aucun lien automatique avec le dépôt forké
Usage typique	Contribuer à un projet open-source sans droits d'écriture	Travailler localement sur un projet

6.2 Merge vs Rebase — Analyse à partir des captures

Les deux captures suivantes illustrent concrètement la différence entre `git merge` et `git rebase` sur le **même** dépôt.

```

PS C:\Users\hp\Documents\Atelier-git> git log --graph --oneline --decorate --all
* 2fcf4c4 (HEAD -> master) Merge branch 'test-branch-2'
| \
| * 4697707 (test-branch-2) Test branch 2
| /
* 4ba7de3 (test-branch) Test branch commit
| * 6138c52 (origin/feature-ajout, feature-ajout) Ajout feature
| /
* 5840b2a (origin/master, origin/HEAD) First change
* 7444d68 Initial commit
PS C:\Users\hp\Documents\Atelier-git>

```

FIGURE 10 – Graphe après `git merge` — historique non linéaire (commit de fusion visible)

```

>> git commit -m "Rebase commit"
[rebase-branch 41436c3] Rebase commit
1 file changed, 1 insertion(+), 3 deletions(-)
PS C:\Users\hp\Documents\Atelier-git> git rebase master
Current branch rebase-branch is up to date.
PS C:\Users\hp\Documents\Atelier-git> git log --graph --oneline --decorate --all
* 41436c3 (HEAD -> rebase-branch) Rebase commit
* 2fcf4c4 (master) Merge branch 'test-branch-2'
| \
| * 4697707 (test-branch-2) Test branch 2
| /
* 4ba7de3 (test-branch) Test branch commit
| * 6138c52 (origin/feature-ajout, feature-ajout) Ajout feature
| /
* 5840b2a (origin/master, origin/HEAD) First change
* 7444d68 Initial commit

```

FIGURE 11 – Graphe après `git rebase master` — `rebase-branch` repositionnée au sommet

Lecture des captures.

- Sur la **figure du merge** (Capture.PNG), le log montre un commit de fusion explicite `2fcf4c4 Merge branch 'test-branch-2'`, avec deux lignes d'historique qui convergent (`| \`). Cette structure en diamant conserve l'intégralité de l'historique des branches.
- Sur la **figure du rebase** (rebase_branch.PNG), la branche `rebase-branch` pointe vers `41436c3 Rebase commit`, qui est placé *au-dessus* du commit de merge de `master`. Le rebase a rejoué le commit de `rebase-branch` après le dernier commit de `master`, produisant un historique **linéaire et sans commit de fusion supplémentaire**.

Merge vs Rebase — Généralisation

	Merge	Rebase
Principe	Crée un commit de fusion qui unit deux branches	Rejoue les commits d'une branche sur une autre base
Historique	Non linéaire — structure en diamant visible dans <code>git log -graph</code>	Linéaire — pas de commit de merge, historique “nettoyé”
Préservation des commits	Commits d'origine conservés tels quels	Les commits sont réécrits (nouveaux SHA)
Sécurité	Sûr pour les branches partagées	Ne jamais rebaser une branche publique/partagée
Usage typique	Intégration de features dans <code>main/master</code>	Mise à jour d'une branche locale avant une PR
Traçabilité	Complète — on voit quand et d'où provient chaque merge	Réduite — l'historique original est réécrit

En résumé : *Merge* privilégie la **traçabilité** et la **sécurité** au prix d'un historique plus complexe. *Rebase* privilégie la **lisibilité** de l'historique en le rendant linéaire, mais nécessite de ne jamais être utilisé sur des branches partagées afin d'éviter des conflits d'historique pour les autres collaborateurs.

7 Conclusion

Ce TP nous a permis de maîtriser l'ensemble du workflow Git collaboratif : de l'initialisation d'un dépôt local jusqu'à la fusion des modifications via une Pull Request sur GitHub. Nous avons également approfondi les concepts avancés de **Fork**, **Merge** et **Rebase**, dont les différences fondamentales ont été illustrées par les captures du terminal et analysées en détail.

La commande `git log -graph -oneline -decorate -all` s'est révélée particulièrement utile pour visualiser l'évolution de l'historique et comprendre visuellement les effets de chaque opération.